Data preparation and analysis for chat model fine-tuning

Michael Wu, Simón Fishman
Aug 22, 2023

Open in Github

This notebook serves as a tool to preprocess and analyze the chat dataset used for fine-tuning a chat model. It checks for format errors, provides basic statistics, and estimates token counts for fine-tuning costs. The method shown here corresponds to the **current fine-tuning method** for gpt-3.5-turbo. See **legacy fine-tuning** for models like babbage-002 and davinci-002.

```
import json
import tiktoken # for token counting
import numpy as np
from collections import defaultdict
```

Data loading

We first load the chat dataset from an **example JSONL file**.

```
data_path = "data/toy_chat_fine_tuning.jsonl"

# Load the dataset
with open(data_path, 'r', encoding='utf-8') as f:
    dataset = [json.loads(line) for line in f]

# Initial dataset stats
print("Num examples:", len(dataset))
print("First example:")
for message in dataset[0]["messages"]:
    print(message)
```

```
Num examples: 5
First example:
{'role': 'system', 'content': 'You are a happy assistant that puts a positive spin on everything.'}
{'role': 'user', 'content': 'I fell off my bike today.'}
{'role': 'assistant', 'content': "It's great that you're getting exercise outdoors!"}
```

Format validation

We can perform a variety of error checks to validate that each conversation in the dataset adheres to the format expected by the fine-tuning API. Errors are categorized based on their nature for easier debugging.

1. **Data Type Check**: Checks whether each entry in the dataset is a dictionary (`dict`). Error type: `data_type` .
2. **Presence of Message List**: Checks if a `messages` list is present in each entry. Error type: `missing_messages_list` .
3. **Message Keys Check**: Validates that each message in the `messages` list contains the keys `role` and `content` . Error type: `message_missing_key` .
4. **Unrecognized Keys in Messages**: Logs if a message has keys other than `role` , `content` , `weight` , `function_call` , and `name` . Error type: `message_unrecognized_key` .
5. **Role Validation**: Ensures the `role` is one of "system", "user", or "assistant". Error type: `unrecognized_role` .
6. **Content Validation**: Verifies that `content` has textual data and is a string. Error type: `missing_content` .
7. **Assistant Message Presence**: Checks that each conversation has at least one message from the assistant. Error type: `example_missing_assistant_message` .

The code below performs these checks, and outputs counts for each type of error found are printed. This is useful for debugging and ensuring the dataset is ready for the next steps.

```
# Format error checks
format_errors = defaultdict(int)

for ex in dataset:
    if not isinstance(ex, dict):
        format_errors["data_type"] += 1
        continue

    messages = ex.get("messages", None)
    if not messages:
        format_errors["missing_messages_list"] += 1
        continue

    for message in messages:
        if "role" not in message or "content" not in message:
            format_errors["message_missing_key"] += 1

        if any(k not in ("role", "content", "name", "function_call", "weight") for k in message):
            format_errors["message_unrecognized_key"] += 1

        if message.get("role", None) not in ("system", "user", "assistant", "function"):
            format_errors["unrecognized_role"] += 1

        content = message.get("content", None)
        function_call = message.get("function_call", None)

        if (not content and not function_call) or not isinstance(content, str):
            format_errors["missing_content"] += 1

    if not any(message.get("role", None) == "assistant" for message in messages):
        format_errors["example_missing_assistant_message"] += 1

if format_errors:
    print("Found errors:")
    for k, v in format_errors.items():
        print(f"{k}: {v}")
```

```
else:
    print("No errors found")
```

```
No errors found
```

Token Counting Utilities

Lets define a few helpful utilities to be used in the rest of the notebook.

```
encoding = tiktoken.get_encoding("cl100k_base")

# not exact!
# simplified from https://github.com/openai/openai-cookbook/blob/main/examples/How_to_count_tokens_with_tikto
def num_tokens_from_messages(messages, tokens_per_message=3, tokens_per_name=1):
    num_tokens = 0
    for message in messages:
        num_tokens += tokens_per_message
        for key, value in message.items():
            num_tokens += len(encoding.encode(value))
            if key == "name":
                num_tokens += tokens_per_name
    num_tokens += 3
    return num_tokens

def num_assistant_tokens_from_messages(messages):
    num_tokens = 0
    for message in messages:
        if message["role"] == "assistant":
            num_tokens += len(encoding.encode(message["content"]))
    return num_tokens

def print_distribution(values, name):
    print(f"\n#### Distribution of {name}:")
    print(f"min / max: {min(values)}, {max(values)}")
    print(f"mean / median: {np.mean(values)}, {np.median(values)}")
    print(f"p5 / p95: {np.quantile(values, 0.1)}, {np.quantile(values, 0.9)}")
```

Data Warnings and Token Counts

With some lightweight analysis we can identify potential issues in the dataset, like missing messages, and provide statistical insights into message and token counts.

1. **Missing System/User Messages**: Counts the number of conversations missing a "system" or "user" message. Such messages are critical for defining the assistant's behavior and initiating the conversation.
2. **Number of Messages Per Example**: Summarizes the distribution of the number of messages in each conversation, providing insight into dialogue complexity.
3. **Total Tokens Per Example**: Calculates and summarizes the distribution of the total number of tokens in each conversation. Important for understanding fine-tuning costs.

4. **Tokens in Assistant's Messages**: Calculates the number of tokens in the assistant's messages per conversation and summarizes this distribution. Useful for understanding the assistant's verbosity.
5. **Token Limit Warnings**: Checks if any examples exceed the maximum token limit (16,385 tokens), as such examples will be truncated during fine-tuning, potentially resulting in data loss.

```
# Warnings and tokens counts
n_missing_system = 0
n_missing_user = 0
n_messages = []
convo_lens = []
assistant_message_lens = []

for ex in dataset:
    messages = ex["messages"]
    if not any(message["role"] == "system" for message in messages):
        n_missing_system += 1
    if not any(message["role"] == "user" for message in messages):
        n_missing_user += 1
    n_messages.append(len(messages))
    convo_lens.append(num_tokens_from_messages(messages))
    assistant_message_lens.append(num_assistant_tokens_from_messages(messages))

print("Num examples missing system message:", n_missing_system)
print("Num examples missing user message:", n_missing_user)
print_distribution(n_messages, "num_messages_per_example")
print_distribution(convo_lens, "num_total_tokens_per_example")
print_distribution(assistant_message_lens, "num_assistant_tokens_per_example")
n_too_long = sum(l > 16,385 for l in convo_lens)
print(f"\n{n_too_long} examples may be over the 16,385 token limit, they will be truncated during fine-tuning
```

```
Num examples missing system message: 1
Num examples missing user message: 1

#### Distribution of num_messages_per_example:
min / max: 2, 9
mean / median: 3.8, 3.0
p5 / p95: 2.0, 6.6000000000000005

#### Distribution of num_total_tokens_per_example:
min / max: 26, 8032
mean / median: 1648.4, 45.0
p5 / p95: 26.8, 4863.6

#### Distribution of num_assistant_tokens_per_example:
min / max: 4, 8000
mean / median: 1610.2, 10.0
p5 / p95: 6.0, 4811.200000000001

0 examples may be over the 16,385 token limit, they will be truncated during fine-tuning
```

Cost Estimation

In this final section, we estimate the total number of tokens that will be used for fine-tuning, which allows us to approximate the cost. It is worth noting that the duration of the fine-tuning jobs will also increase with the token count.

```
# Pricing and default n_epochs estimate
MAX_TOKENS_PER_EXAMPLE = 16385

TARGET_EPOCHS = 3
MIN_TARGET_EXAMPLES = 100
MAX_TARGET_EXAMPLES = 25000
MIN_DEFAULT_EPOCHS = 1
MAX_DEFAULT_EPOCHS = 25

n_epochs = TARGET_EPOCHS
n_train_examples = len(dataset)
if n_train_examples * TARGET_EPOCHS < MIN_TARGET_EXAMPLES:
    n_epochs = min(MAX_DEFAULT_EPOCHS, MIN_TARGET_EXAMPLES // n_train_examples)
elif n_train_examples * TARGET_EPOCHS > MAX_TARGET_EXAMPLES:
    n_epochs = max(MIN_DEFAULT_EPOCHS, MAX_TARGET_EXAMPLES // n_train_examples)

n_billing_tokens_in_dataset = sum(min(MAX_TOKENS_PER_EXAMPLE, length) for length in convo_lens)
print(f"Dataset has ~{n_billing_tokens_in_dataset} tokens that will be charged for during training")
print(f"By default, you'll train for {n_epochs} epochs on this dataset")
print(f"By default, you'll be charged for ~{n_epochs * n_billing_tokens_in_dataset} tokens")
```

```
Dataset has ~4306 tokens that will be charged for during training
By default, you'll train for 20 epochs on this dataset
By default, you'll be charged for ~86120 tokens
```

See **https://openai.com/pricing** to estimate total costs.